Stoquify codebase rewrite: roadmap and execution guide

Prepared **27 September 2026**, using the **24 September 2026** rewrite reports, engineering assessment, execution-method prompt and later POS review.

This report explains how to execute the remaining program. Its repository inspection is read-only. Application implementation, tests, builds, browser sessions, migrations and deployment were not performed for this report.

1. Main recommendation

Continue the existing **domain-oriented modular monolith**, preserving Next.js, TypeScript, Prisma/PostgreSQL and authoritative domain services. Review the complete maintained product, but replace code only where behavior, security, maintainability or measured performance evidence justifies it. A reviewed and verified decision to retain an implementation is a valid outcome.

The immediate milestone is **reliable verification, complete scope discovery and an actionable technical backlog: stages 00–02**. Then complete shared foundations and domain workflows in dependency order.

The central operational gap is that the published execution-method prompt requests a work register, evidence index, handoff and validated operating procedure, but its five named files do not exist in the prescribed source folder. Establish that operating set before broad implementation. The companion execution-plan.json supplies proposed starting packages; it is a planning artifact, not an active program register or a completion claim.

Success means preserved and verified business workflows, clearer ownership, measured improvements and satisfied completion gates. Deleted lines, large passing-test counts and rearranged folders cannot establish that outcome alone.

2. Which reports were found

The latest matching codebase rewrite reports are dated **24 September**, three days before this request. The related modularity proposal is dated **20 September**, approximately a week earlier. Their content dates are more reliable than publication-copy modification times.

Source	Role in this guide
Complete execution roadmap	Primary eleven-stage continuation plan, dated 24 September.
Final rewrite report and verification record	Implemented work, measurements and verification limits.
Baseline, initial rewrite plan, progress and reviewer coverage	Counting rules, acceptance budgets, review responsibilities and retained scope.
Execution-method prompt	Proposed operating method. Publication did not execute its instructions.
Engineering approach review	Architecture recommendation and bounded adoption pilots. Its five adoption steps fit inside the larger roadmap; they do not replace stages 00–10.
Later POS gate review	Updated 24 September at 09:15 UTC; revises the earlier inventory/build blocker picture.

Source	Role in this guide
Modularity roadmap	20 September product-composition proposal supporting existing domain ownership. Its purchasing UX experiment has a separate scope.

An additional successful build diagnostic was found from **27 July**. It predates the September changes and cannot establish a current release build.

3. Current position and evidence freshness

The current observed commit is **62a56036a5ac9b37236a20c9b148052199069479**, dated 25 September. Earlier evidence described an older commit plus working-tree edits, so commit identity alone cannot establish evidence equivalence.

This inspection compared file bytes with saved SHA-256 records:

- All **four application files and three test files** in the rewrite change manifest still match.
- All **22 entries** in the later POS gate source manifest also match.
- The execution-method artifacts requested in the published prompt remain absent from the canonical source folder.

See source-review.json. These matches preserve useful provenance; they do not prove unchanged transitive dependencies, all scanned producers, runtime configuration, database contents or external approvals.

Area	Last recorded outcome	What execution should do
Cash/payment-history client	Cache identity includes actor, organization, permissions and filters; foreign responses and late export results are guarded	Preserve the implementation and finish authenticated browser verification.
Cash-history summaries	Grouped decimal sums/counts replace two unbounded summary reads	Verify real SQL, full endpoint cost, concurrency and filter equivalence.
Cash-shortage preflight	106 wrappers consolidated; 336 exports retained; 5,610 serialized comparisons matched	Preserve the code and contract corpus. Do not repeat the one-time transformation.
Application reduction	1,745 attributable lines removed across four application files	Historical measured delta, not a whole-platform completion percentage.
Rewrite tests/typecheck	1,019 tests passed across six suites; 11 hook tests rerun; final typecheck passed	Historical scoped results. The 11 are repeated tests, and persistence in the six suites is mocked.
Inventory boundary	Earlier rewrite: 20 findings. Later POS review: zero active findings, with 24 fixture callsites dispositioned	Refresh current scan/input coverage; do not treat the earlier 20 as unaddressed without checking the later evidence.
Build	Rewrite run timed out. Later POS run compiled, validated types and generated nine static pages, then failed standalone packaging with Windows EPERM	Latest known gap is a successful packaged artifact. Investigate current packaging/host behavior; extending the old timeout alone is insufficient.
Browser	Earlier probes timed out. Later French drawer review passed limited functional checks; cashier access remained unavailable	Preserve the French result at its measured 927×521 viewport. It does not establish cash-history identity tests, mobile coverage or cashier checkout.

Area	Last recorded outcome	What execution should do
Application postingCycle migration	Exact approved local migration completed on 24 September	Do not repeat it as an assumed rewrite prerequisite. Verify schema/history in the chosen test environment.
POS governance	G1: 13/13 structural, 0/11 authentic decisions. Program: 12/12 structural, 0/10 gates, 0/9 external evidence	Continue eligible engineering work; retain release blockers for authentic decisions, evidence and candidate alignment.
Execution controls	execution-method.md, work-packages.json, evidence-index.json, handoff.md and method-validation.md absent under what-next/codebase-rewrite	Create one canonical operating set and derive summaries from it.

The original baseline measured **1,602 application files, 383,365 physical lines, 102 direct production dependencies, 62 repeated-body groups and 51 files absent from TypeScript root membership**. Those are September 24 measurements under stated counting rules, not fresh September 27 totals. A root exclusion does not establish exclusion from transitive typechecking.

Seven of eight original source-import cycle groups disappeared during transpilation. One accounting/inventory group retained an emitted-code cycle, without proving an initialization defect. Review that surviving cycle on its merits; do not add architectural layers to fix erased type-only cycles.

4. Extracted roadmap: all eleven stages

Ownership below names responsibilities, not assigned people or completed approvals.

Stage	Work and lead responsibility	Completion evidence
00 — Reliable verification	Platform/SRE, QA and data: accepted snapshot, build, app, dedicated data/identities and current gate dispositions	Complete packaged artifact, responsive test environment, verified first slice and explicit remaining release dependencies.
01 — Whole-product inventory	Architecture, product and domain owners: maintained areas, entry points and supported workflows	Complete scope/contract/owner/dependency registers; no unexplained application areas.
02 — Architecture and targets	Technical lead and reviewers: ownership, dependency direction, budgets and packages	Recorded decisions, measured targets and ordered backlog with acceptance/rollback requirements.
03 — Shared foundations	Platform/security/frontend: tenancy, RBAC, modules, cache scope, contracts, errors, events and shared presentation	Compatible consumers and positive/negative boundary tests; authoritative business controls preserved.
04 — Inventory, POS and offline	Inventory/POS owners, finance and QA	Stock/value conservation, sale/tender/drawer/receipt/return behavior, idempotency, concurrent execution, replay and recovery.
05 — Commercial and payments	Purchasing, customer/AR, payment and reconciliation owners	Order/receipt/invoice/credit lifecycles, approvals, settlement/suspense and financial tie-outs.
06 — Accounting, close and statutory	Accounting, controls and qualified country reviewers	Balanced posting, periods, corrections, invalidation, immutable evidence and dated country rules.
07 — HRIS and payroll	HRIS/payroll, privacy and finance	Approved effective-dated inputs, frozen snapshots, precise calculations, approvals, corrections and ledger integration.

Stage	Work and lead responsibility	Completion evidence
08 — Remaining product areas	Respective product/domain owners	Analytics, dashboards, onboarding, admin/media, modules/billing, referrals, communications, AI/agents and inventory-discovered workflows.
09 — Optimization and retirement	Performance/platform and affected owners	Comparable measured improvements; consumers migrated; obsolete implementations/dependencies retired; adapters accounted for.
10 — Integrated acceptance and handover	QA/release, security, operations and accountable domain reviewers	Complete scope reconciliation, integrated journeys, required gates, recovery evidence and usable support handover.

Dependencies and critical path

Stages 00 and 01 can progress concurrently. Discovery does not need a successful production build. Stage 02 uses the behavioral map and reliable measurements; broad implementation follows relevant shared contracts and acceptance budgets.

Default progression: **00/01 → 02 → 03 → domain packages in 04–08 → 09 → 10**. This is a dependency framework, not a demand to finish every inventory feature before inspecting payroll. Independent discovery, test design and contract review can advance across domains.

Understand accounting posting and close-invalidation contracts in stages 01–03 because inventory/POS changes consume them. Stage 06 is the deeper accounting implementation wave. Similarly, settle module admission and retained-work/recovery semantics before changing offline or subscription behavior.

Two paths govern completion:

1. **Engineering:** reproducible environment → workflow inventory → contract/target decisions → verified slices → integrated acceptance.
2. **Release evidence:** accountable decisions/candidate identity → provider/country/hardware/operational evidence → release authorization.

Start external evidence preparation early. Local passing tests cannot replace missing external facts or approvals.

5. First milestone: twelve concrete packages

The companion JSON preserves these IDs and dependencies. None is marked complete by this report.

ID	Scope, owner and prerequisites	Required output / definition of done
RW00-01	Engineering lead: accepted source/environment snapshot. Start now; preserve tracked and untracked work.	Source/config/lockfile hashes, runtime/package-manager record, owned resources, evidence freshness and differences from earlier inputs.
RW00-02	SRE/build owner; after RW00-01. Safe build wrapper, diagnostics, Next config and CI.	Reproduce/classify current failure; complete standalone artifact; successful start/smoke check; output path, size, exit/logs and configuration provenance.
RW00-03	QA/data/security; after RW00-01. Dedicated database, fixtures, auth and browser configuration.	Verified database identity/schema; authorized, denied, cross-tenant and module-disabled identities; responsive app and owned port/output.

ID	Scope, owner and prerequisites	Required output / definition of done
RW00-04	Controls/platform; after RW00-01. Inventory/service/POS governance tooling and records.	Fresh scoped gates; individual current findings dispositioned; compatible prior fixture evidence reused; authentic approval gaps retained as release dependencies.
RW00-05	Data/backend and QA; package closure after RW00-02/03. Cash-history UI/hook/action/service.	Real PostgreSQL equivalence/plans; full endpoint measurements; authenticated scope/revocation/export cases; localized keyboard/recovery evidence.
RW00-06	QA/integration; after RW00-01. Cash/preflight regression corpus.	Relevant regression/type/schema checks pass on the candidate; exports/serialized outputs remain compatible; no historical evidence overwritten.
RW01-01	Architecture/product: discover all first-party surfaces independently of build work.	Routes/actions/APIs/jobs/events/webhooks/scripts/schema/CI/providers/tests/legacy scope register; bind final results to RW00-01 snapshot before milestone closure.
RW01-02	Domain owners; after RW01-01. Required business/user journeys.	Actor/authority/state/data-owner/consumer contracts, failure/recovery paths and acceptance tests.
RW01-03	Architecture/QA; after RW01-01. Duplication, emitted imports, effective coverage and consumers.	Current disposition of historical repeated groups/cycles/non-root files; source reachability established before any removal.
RW02-01	Architecture/security and owners; after RW01-02/03.	Target architecture, explicit transaction/domain ownership, neutral contracts, provider interfaces and decision register.
RW02-02	Performance/QA/product; after RW00-05/06 and RW01-02.	Measured target ledger with fixed datasets/environment, variance, commands and correctness/performance/size budgets.
RW02-03	Lead/integration reviewer; after all other initial packages.	Canonical register, evidence index, procedure, handoff and method validation; all discovered areas mapped; next shared-foundation package unambiguous.

DB characterization within RW00-05 may begin once RW00-03 is available; its complete artifact/browser evidence depends on RW00-02 as well. Missing external release approval does not prevent unrelated discovery, but any required gate remains explicitly blocking for its actual release scope.

Build and environment work

Use the later packaging failure as the latest diagnosis, while retaining the earlier timeout. Record effective symlink capability, dependency tracing, output ownership and resource contention. Choose a supported environment with required filesystem capabilities. A clean CI/Linux build is a candidate, provided it includes the accepted source and compatible dependencies/runtime.

Do not silently alter machine security settings or disable standalone packaging to manufacture a pass. Compilation is insufficient: start and smoke-test the resulting artifact.

Provision a dedicated named test database and controlled identities. Verify the connected database before migrations/seeds. A personal browser session or an earlier promise to switch accounts is not a reproducible test fixture. Establish cashier/manager, denied, cross-tenant, location and module-disabled scenarios deliberately.

Cash-history data and browser proof

Compare exact before/after outputs with fixed data covering empty results, multiple tenants/locations/actors, own/manager visibility, filter/date boundaries, opening/closing balances, corrections/refunds, tender

methods/statuses and Decimal edge cases. Retain bounded detail pagination, cursor binding and redaction.

Measure plans, rows transferred, query count/time, database resource use and full endpoint p50/p95 at fixed cache/concurrency settings. The historical 200,000-to-49 row reduction was a mocked transport observation; four queries remained four, and PostgreSQL aggregation still has a cost.

Browser scenarios must cover actor/tenant/permission changes, revocation, foreign responses, stale export completion, filters/pagination/selection, empty/error/retry states and accessible feedback. Verify EN/FR at measured viewports plus relevant keyboard/zoom/assistive-technology conditions. Existing transaction-history tests must be mapped to these requirements; their project name alone does not establish coverage.

Use identified disposable fixtures for financial mutations. Rendering a page does not prove posting correctness.

Inventory and governance proof

Run fresh scanners with new output paths. Matching scanner hashes does not show that every scanned producer is unchanged. Review new findings individually; preserve the earlier callsite dispositions where still applicable.

Track authentic decision/issuer requirements separately. Do not fabricate approvals, update a stale candidate just to silence checks, or widen fixture exemptions until a scan passes.

6. Architecture and the first bounded implementation candidates

Keep actions/APIs responsible for request validation, trusted actor/tenant identity, permissions and module admission. Services retain authoritative business behavior and public validation. Keep sale/stock/posting/audit/outbox transaction composition explicit. Hooks cache scoped read models; they do not become another business authority.

After prerequisites, use these candidates from the engineering assessment:

1. **Make the current standard accurate.** Reconcile ADR examples with the protected-action calling convention and public-service validation. Document the authoritative package manager/runtime. Current CI uses npm ci with Node 20; historical local evidence used Node 22.17.0; npm and pnpm lockfiles both exist. Inventory effective TypeScript/service-test coverage before changing exclusions or coverage configuration.
2. **Extract one offline admission policy seam.** Characterize the deterministic decision block in services/pos/offline-sync.service.ts; extract explicit inputs/time without new database/session dependencies. Preserve precedence, messages/codes, hashes, transaction scope and replay behavior. Test expiry immediately before/at/after its boundary, snapshots, signatures, gaps, hash failures and simultaneous failures. Do not enable offline deployment as part of extraction.
3. **Give finance export contracts a neutral owner.** The current export service still imports from components/DataTableComponents/finance-accounting-contracts. Move shared runtime schemas/constants/types to an agreed headless contract location, update consumers and retain compatible re-exports only where required. This is ownership/dependency debt; the existing contract file is headless TypeScript, not evidence that React executes inside the service.
4. **Enforce dependency direction.** Inventory existing findings, then add an AST-aware rule for new forbidden value imports with individual exceptions and removal conditions. Distinguish type-only edges from runtime dependencies.

5. **Reduce concentrated responsibilities by operation.** Candidate owners include payroll-control.service.ts, ap-control.service.ts, close-assurance.service.ts and pos.service.ts. Select using change pain, incidents, control risk and traceability, not line count alone.

No evidence here justifies replacing the framework, introducing microservices per module, another ledger/entitlement engine or pervasive event sourcing. Revisit infrastructure boundaries only after workload or independent-availability evidence and a credible consistency/recovery model establish benefit.

7. How to execute the domain waves

Each slice includes its real UI → hook → action/API → service → persistence → event/provider consumers. A service-only refactor is incomplete if operator workflows or downstream reconciliation break.

Wave / principal paths	Workflow order and invariants	Required evidence
Inventory/POS/offline: services/inventory, services/pos and related actions/hooks/components	Setup → receipt/valuation → sale/tender/drawer → return/correction → replay/end-of-day. Tenant/location scope, stock/value conservation, atomic writes, original-request recovery and receipt failure semantics.	Real DB concurrent sale/count/transfer/return; duplicate/interrupted replay; no duplicate money/stock/posting effects; EN/FR operator recovery.
Commercial/payments: purchasing, purchase-order, supplier, customer, order-to-cash, payments, reconciliation	Order → receiving → invoice/match → approval → payment → settlement/suspense → reconciliation/correction.	Maker-checker/denied cases, partials/credits/cancellations, duplicate/out-of-order provider events, monetary tie-outs and controlled provider contracts.
Accounting/close: accounting, compliance, regulatory, evidence	Source event → balanced posting → period/close evidence → correction/invalidation → accountant review.	Closed-period protection, immutable lineage, upstream correction propagation, recovery and dated country provenance/qualified review.
HRIS/payroll: hris, payroll, employee self-service	Employee/contract/time/compensation → approved snapshot → calculation → approval → payment/declaration → ledger/close.	Effective dates, Decimal calculations, correction lineage, employee/manager privacy, segregation of duties and payroll-to-ledger tie-out.
Remaining areas: analytics/BI, dashboards/command centers, modules, organization/auth/roles, onboarding/media, communications/referrals, AI/agents/signals	Inventory-defined user, admin, support and background workflows.	Canonical KPIs, scoped/fresh reads, signed/deduplicated billing transitions, retained obligations after downgrade, upload/export privacy and agent authority/denial limits.

Integrated journeys should include:

- Buy → receive → invoice → approve → pay → reconcile → close.
- Sell → tender → stock/posting → receipt → return/correction → close invalidation.
- Offline capture → conflict/replay → final posting and acknowledgement.
- Employee change → approved payroll → payment/declaration proof → ledger.
- Module/role change while accepted work remains pending.

These are proposed acceptance journeys, not a claim that every supported workflow has already been discovered. Inventory callbacks, imports/exports, operator scripts, scheduled work, integration users and dynamic registrations explicitly.

8. Work-package contract and repeatable procedure

Maintain one canonical what-next/codebase-rewrite/work-packages.json. Derive status summaries from it.

Disposition: retain, consolidate, replace, remove.

Progress: unreviewed → inventoried → characterized → ready → implementing → verifying → verified. A blocked package records its previous state and missing prerequisite. Retaining code is not proof that it is verified.

Each package records: ID/stage/outcome; exact files/contracts owned; consumers/exclusions; dependencies; implementation/review/integration responsibilities; risk and invariants; source/environment baseline; measurable acceptance; commands/fixtures; migration/rollback; temporary adapters/removal conditions; evidence; status/blocker/next action.

Execution loop:

1. Trace behavior, authority and all callers, including dynamic/background entry points.
2. Separate required behavior from reproducible defects. Bind the baseline and set acceptance budgets before editing.
3. Add focused characterization where evidence is missing.
4. Implement the smallest coherent change across affected boundaries.
5. Verify successful and denied behavior, financial/data invariants, failure/recovery, concurrency and consumer compatibility; measure the changed performance boundary.
6. Migrate consumers and remove superseded paths only after static/dynamic entry-point checks. Assign an owner/removal condition to every temporary adapter.
7. Review the final diff and evidence against final source/configuration hashes.
8. Close only when all mandatory package criteria pass; otherwise record the exact next action and continue an independent eligible package.

Ready requires satisfied implementation prerequisites and named ownership. Verified requires valid acceptance evidence, compatible consumers and accounted-for adapters. A failing tenant, authority or financial-invariant test prevents closure even when performance and size improve.

9. Ownership, concurrency and resume

Use one integration owner for shared contracts and source coordination. Separate implementation and review responsibilities. Where independent or qualified review is required, assign real reviewers and retain authentic decisions. Multiple review perspectives do not themselves create independent approvals.

Apply the ten core responsibilities from reviewer coverage: platform architecture; backend/domain/integration; database/migrations; security/IAM/privacy; frontend/design systems; workflow UX/accessibility/localization; product/business process; quality/release; SRE/performance; and SaaS packaging/customer operations.

Activate its twelve domain responsibilities as relevant: finance/controls; statutory/country packs; audit/data quality; POS/inventory/offline; purchasing/AP; HRIS/payroll/privacy; payments/settlement; accounting close; analytics; AI safety; API/webhook/outbox/import/export; and change/support/rollout.

Start with **two active work packages**: an environment/verification lane and a discovery/contracts lane. This is a proposed control for the shared checkout and observed build contention, not a measured optimal team size.

Initially serialize heavy builds, whole-project typechecks, benchmarks and database-mutating suites on the shared host.

Parallel implementation requires disjoint file/contract ownership, integration responsibility, and distinct ports, output directories, browser contexts and fixture identities. A checkout from HEAD may omit accepted uncommitted work: prepare the actual candidate snapshot first. Describing several responsibilities does not require creating agents or new tasks.

At every package boundary/interruption, update handoff.md with:

- Last verified source and current unfinished diff.
- Commands completed, running processes and resource ownership.
- Environment identities, valid/stale evidence and unresolved decisions.
- Blockers, adapters and exact next action with prerequisites.

On resume, compare that record with current files and processes. Reuse still-valid results; rerun only affected checks. Stop/clean only resources owned by the run.

10. Verification commands and boundaries

These commands were checked against current repository scripts/configuration. They are proposed for the execution run, not run as application checks for this report. Use a fresh evidence directory and retain actual start/end times, exit codes and scope.

Source and regression checks

```
npm run typecheck
npm run prisma:validate

node node_modules/jest/bin/jest.js --config what-next/codebase-rewrite/evidence/jest.config.cjs --
runInBand --runTestsByPath hooks/__tests__/useCashPaymentHistoryWorkbench.scope.test.tsx
hooks/__tests__/useCashPaymentHistoryWorkbench.test.ts services/pos/__tests__/cash-payment-
history.service.test.ts components/finance/__tests__/CashPaymentHistoryWorkbench.test.tsx
services/leakage/__tests__/pos-cash-shortage-production-activation-preflight.test.ts
actions/pos/__tests__/cash-payment-history.actions.test.ts
```

Run serially when resource use requires it. Inspect changed test setup/import side effects first. Typecheck and schema validation do not establish whole-product behavior or apply a migration.

For scanners, substitute an existing new evidence directory for RUN_DIRECTORY:

```
node scripts/inventory-boundary-gate.js --mode fail --out RUN_DIRECTORY/inventory.md --json-out
RUN_DIRECTORY/inventory.json
node scripts/service-boundary-gate.js --mode fail --out RUN_DIRECTORY/service.md --json-out
RUN_DIRECTORY/service.json
node scripts/pos-g1-contract-gate.js --mode fail --no-write
node scripts/pos-enterprise-program-gate.js --mode fail --no-write
```

RUN_DIRECTORY is a placeholder, not a prescribed literal folder. Capture governance stdout and exits separately without rewriting frozen decision records. The scanners write report artifacts.

Safe build pattern

After source/resource ownership is settled, choose a fresh direct-child output directory. Preserve previous environment and generated configuration values for restoration.

```
$env:NEXT_DIST_DIR = '.next-rewrite-verification-UNIQUE_RUN_ID'
$env:BUILD_APP_SKIP_CLEAN = '1'
$env:BUILD_APP_TIMEOUT_MS = '3600000'
$env:NEXT_TELEMETRY_DISABLED = '1'
node scripts/build-app-safe.js --dry-run --skip-clean
```

Only after reviewing the dry-run directory/process policy, execute **npm run build:app** with the same variables. The illustrated 60-minute bound accommodates earlier diagnostics; it is not a fix. The inspected diagnostics wrapper can capture logs/timing, but inspect the actual isolated artifact rather than assuming a default BUILD_ID path is correct.

Restore only run-owned generated configuration/environment changes afterward. Do not delete another task's build or dev output.

npm run build is different: current scripts run release-secret preflight and migration deployment before the linted build. verify:ci and policy:gates also require inspection and a prepared environment. The policy aggregate includes PostgreSQL runners, runtime/evidence checks and report writes. Run applicable understood checks first, then the complete required bundle in its controlled environment. Skipping a mandatory check still prevents release.

Database and browser execution

The workflow:assurance:runtime-check command inspects a database. Verify its intended connection first and use the underlying script's --out/--json-out options for new evidence. Table/migration presence is not complete history certification.

The existing test:e2e:transaction-history npm command deploys migrations and seeds before browser execution. Do not point it at the application database by default. After test-environment provisioning and side-effect review, the lower-level project is:

```
node node_modules/@playwright/test/cli.js test --project=transaction-history-authenticated-release
```

It still runs an authentication dependency and may start the configured server. Set the verified test base URL/port/storage state; skip web-server startup only when the intended server is running. Protect auth-state files. Map actual coverage before adding the missing identity-change or cashier scenarios.

Historical inspect.cjs writes into its evidence folder, benchmark.cjs overwrites benchmark.json, and the consolidation script is intentionally a one-time transformation. Preserve those originals. Give new measurements a separately configured output location.

Evidence reuse

Record package/run ID; relevant source/dependency/config identity; command; environment/dataset; timing/exit; artifacts; and limits. Use **passed, failed, timed out, blocked, not run** distinctly.

Code, schema, fixtures, dependencies, material configuration or policy/approval changes invalidate affected results. Matching seven source files is insufficient to reuse a whole-tree build verdict.

11. Measurements, schedule and progress

Prioritize correctness/security, then clarity, performance and size. Establish budgets from representative baselines before each replacement.

Dimension	Measure
Behavior	Verified workflow/contract coverage; actor/tenant/module denials; exact money/stock/ledger invariants; failure/recovery.
Maintainability	Repeated authoritative logic, unclear ownership, emitted cycles, boundaries touched by a representative change and reviewer traceability.
Performance	Fixed dataset/environment/cache/concurrency; SQL plans/query cost/rows; full endpoint p50/p95; throughput/memory/UI responsiveness/bundles.
Size	Same original counting rules; include new helpers/dependencies; separate tests, snapshots, generated files and concurrent changes.
Operations	Outbox/retry age, failure/conflict rates, recovery and support diagnosis; product-specific objectives grounded in actual observations.

Report verified workflows/packages with remaining risk. Do not publish a completion percentage until the scope denominator and weighting are meaningful.

The engineering review estimated **1–3 engineer-days** for standards, **2–4** for the bounded offline policy pilot, and **2–5** for subsequent pattern/contract enforcement, assuming familiarity and review capacity. These are historical narrow estimates, not a schedule for the whole rewrite.

Re-estimate after stages 00–02 from discovered backlog, dependencies, capacity, external waits and measured throughput. Source-line counts do not justify a complete-platform delivery date.

A useful first five-working-day planning horizon is:

1. Bind source/resources and start whole-product discovery.
2. Reproduce packaging; provision test data and identities.
3. Run focused source gates and begin real cash-history characterization.
4. Consolidate the first workflow/consumer maps and architecture decisions.
5. Review available milestone evidence and choose the next eligible queue.

This is a sequencing target, not a promise that packaging, access or complete inventory will finish in five days.

12. Migration, rollback and release decisions

For code-only changes, retain public contracts and support selective reversal of the attributable patch. For schema changes, plan compatible expansion, versioned readers/writers, scoped backfill, count/balance reconciliation, consumer transition, recovery rehearsal and later retirement.

Keep source rollback distinct from business-data recovery. Use established compensating operations for completed financial effects; destructive resets are not a rollback strategy.

For each pilot identify tenant/role/operation scope, measured stop conditions, support ownership and the real mechanism for stopping new work while retaining accepted obligations/history. Do not assume a suitable switch exists; new control behavior needs review.

Reuse the prepared POS decision packets and nine external-verification procedures. Assign real owners, establish the trusted issuer/public-key process, bind signatures to the reviewed candidate/contract, and obtain authentic provider/country/hardware/operational evidence.

An inventory scan pass does not approve inventory/returns policy, certify statutory behavior or authorize deployment. Program engineering acceptance and production authorization need separate recorded decisions.

13. Validate the operating method and completion

During RW02-03, use actual package IDs and simulated register states to exercise:

Scenario	Required behavior
RW00-01 closes with valid evidence	RW00-02/03/04/06 become eligible subject to resources/authority; discovery may already be running.
Packaging remains blocked	RW00-02 remains blocked while independent RW01/data/gate work continues.
Authority or monetary invariant fails	Affected package cannot be verified despite performance/size success.
A source/configuration input changes	Invalidate affected evidence; preserve originals and select relevant reruns.
Shared-file/resource conflict	Serialize or establish isolation before implementation proceeds.
Interrupted work resumes	Reconcile handoff with actual files/processes; avoid duplicate commands.
Retained module lacks evidence	Keep it unverified; disposition is separate from status.
One required final gate is unresolved	Complete/ready claim remains blocked; valid unrelated results are preserved.

These are proposed method-validation scenarios, not application tests performed in this report. The companion plan is structurally checked for unique IDs, valid dependencies, acyclicity and representation of stages 00–10.

The whole program is complete only when every maintained area has a verified disposition; required workflows and consumers are protected; authority/data/financial invariants pass; the candidate has a runnable packaged artifact; required database/browser/accessibility/localization/security/recovery checks pass; targets are met or explicitly reconciled; superseded paths/adapters are retired or owned; and current evidence, source identity and operational handover agree.

Required professional or operational approvals remain required. A document, local scan or test count cannot substitute for them.

Start with RW00-01 and independent RW01-01 discovery. Assign integration ownership, bind the accepted snapshot, establish the canonical register, reconcile the later POS evidence, and finish the existing cash-

history/preflight slice through RW00-02–06 before expanding implementation. Choose the first shared-foundation pilot only after its contracts and budgets are ready.

14. Verification performed for this report

Read-only inspection located/reconciled the reports, examined current scripts/CI/browser configuration, compared saved source hashes, checked missing operating artifacts, and confirmed cash-history scope/grouping and the finance-export dependency.

Historical graph node hooks_usecashpaymenthistoryworkbench_usecashpaymenthistoryworkbench is in community 7; pos_cash_payment_history_actions_getcashpaymenthistoryaction is in community 79. These graphs assist navigation and do not certify current relationships.

No application suite, production build, browser login, database query/migration, provider test or deployment was executed. Recorded application results remain historical or planned as labelled. Report/plan/PDF checks concern publication integrity only.

Source references